AI INFRASTRUCTURE

kagent Baseline Test

2026-11-10 Author: Ciaran Donegan kagent v0.10.6 Status: Continuous

Fleet footprint baseline for the kagent agent framework on a 5-node bare-metal cluster. 102 pods deployed concurrently across 6 agent types (17 per type). Idle memory and CPU measured from Prometheus with all pods running simultaneously. Establishes the per-pod resource floor for capacity planning a 100-agent fleet on EKS.

All Go agent types converge at approximately 7 MB idle RSS regardless of system prompt size, tools, or conversation state. Python ADK pods idle at 219 MB -- 32x more. Linear scaling confirmed for 3 of 6 types with full 17-pod concurrent data. Worker node scheduling limits (CPU request ceiling, 110 pods per node) prevented full deployment for the remaining 3 types; per-pod figures are consistent with the Go runtime baseline.

Test Environment

Local 5-node bare-metal Kubernetes cluster: 2 schedulable worker nodes (wk1: 4 CPU, 10.5 GB; wk4: 12 CPU, 38.8 GB) and 3 control plane nodes with untolerated taints. kagent installed via Helm from the upstream OCI chart. Model gateway: LiteLLM (proxying Claude Sonnet 5.5 via the Anthropic API).

All 6 agent deployments scaled to 17 replicas simultaneously (102 target pods total) in namespace `kagent`. Prometheus memory and CPU scraped after a 13-minute settle window once pods reached Ready. Worker node scheduling limits (CPU request exhaustion and 110-pod-per-node ceiling) prevented full deployment: 77 of 102 pods reached Ready. go-long-context (8 pods) and go-multiturn (2 pods) are partial samples; per-pod figures are consistent with the Go runtime baseline from the other 4 types.

Key Figures

102-POD FLEET MEMORY

4.3 GB

Mixed fleet idle -- 17 pods per type, all 6 types concurrent

PYTHON VS GO MEMORY

32x

219 MB vs 7 MB per pod at idle

GO RUNTIME IDLE

~7 MB

All 5 Go types converge -- system prompt size and tools have no effect on idle memory

100 PYTHON-MINIMAL PODS

21.9 GB

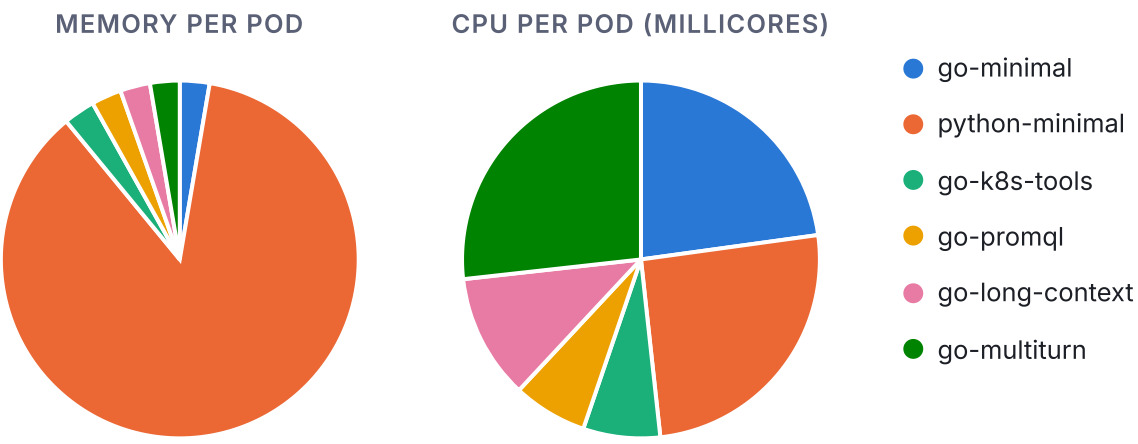
vs 680 MB for 100 go-minimal pods

Fleet Footprint

All 6 agent types scaled to 17 replicas concurrently (102 target pods). Prometheus memory and CPU scraped at idle after a 13-minute settle window. The scale test hit worker node limits at 77 pods -- go-long-context and go-multiturn are partial samples (8 and 2 pods respectively); per-pod figures for both are consistent with the Go runtime baseline.

AGENT TYPE	RUNTIME	PODS	MEMORY / POD	CPU / POD (M)	100-POD MEMORY	100-POD CPU (M)
`go-minimal`	Go	17	6.8 MB	0.88	680 MB	88
`python-minimal`	Python	17	218.8 MB	0.99	21.9 GB	99
`go-k8s-tools`	Go	17	7.2 MB	0.27	720 MB	27
`go-promql`	Go	16	6.9 MB	0.26	690 MB	26
`go-long-context`	Go	8 *	6.9 MB	0.44	690 MB	44
`go-multiturn`	Go	2 *	6.8 MB	1.04	680 MB	104

* Partial sample -- worker nodes hit CPU request exhaustion and 110-pod-per-node ceiling at 77/102 pods. Per-pod figures measured from running pods only.
Idle-only: pods were not serving requests during measurement. Memory figures are RSS. CPU figures are Prometheus rate over a 2-minute window.



Model Compliance

The same 6 agent types run against two model backends. Only the model changed between runs.

AGENT TYPE	OLLAMA (LOCAL)	CLAUDE SONNET 5.5
`go-minimal`	input-required all 17 runs. Ollama asked for clarification instead of answering.	completed all 17 runs
`python-minimal`	input-required all 17 runs	completed all 17 runs (token counts absent, ADK limitation)
`k8s-tools`	12/17 completed, 2 input-required. High variance: 29ms–219s. Hit 4,096 token context ceiling on every completed run. Output tokens ranged 16-1,842. Model was unpredictable.	completed all 17 runs. Avg 6.3s. Consistent 690 output tokens.
`promql`	input-required all 17 runs	completed all 17 runs
`long-context`	input-required all 17 runs	completed all 17 runs
`multiturn`	input-required all 17 runs	12/17 completed, 5 input-required (intentional ambiguity in prompt, correct Claude behaviour)

Ollama returned `input-required` on 95 of 102 runs. This is a system prompt compliance failure, not a kagent issue. The local test uses llama3 8B on a single GPU via llama.cpp, which is not representative of the production inference stack (vLLM, larger models, EC2 G6e). A meaningful open-model comparison requires EKS access.

Findings

1. Go vs Python runtime memory

All 5 Go-runtime agent types idle at 6.8-7.2 MB RSS regardless of system prompt size, tool definitions, or conversation state. Python-runtime pods idle at 219 MB -- 32x more than go-minimal. The difference is the Python interpreter and google-adk package overhead, present in every pod regardless of what the agent does. At 100 pods, Go costs 680-720 MB; Python costs 21.9 GB. The kagent controller itself is always Go; the runtime choice only affects each agent pod.

2. Tool-use cost and latency

The k8s-tools agent uses 3,263 prompt tokens vs 642 for a minimal agent -- 5x more prompt tokens, 5.8x more total cost per call. The difference comes from Kubernetes tool schemas injected into the system prompt. Latency averaged 6.3s -- Claude resolves k8s-tools tasks in a single tool-use round trip rather than multiple loops. All 17 runs completed successfully. At idle, go-k8s-tools memory (7.2 MB) is indistinguishable from go-minimal (6.8 MB) -- the tool schema cost is in tokens, not pod footprint.

3. Tool-call compliance: Claude vs Ollama

All Go agents completed with zero tool-call errors against Claude Sonnet 5.5. The go-multiturn agent returned `input-required` on 5 of 17 runs. This is correct behaviour for an intentionally ambiguous prompt. Against Ollama, 95 of 102 runs returned `input-required` regardless of agent type. This is a model compliance issue, not a framework issue. Ollama could not follow the system prompt instructions. See the Model Compliance section above.

Gaps vs Production Stack

MISSING COMPONENT	PRODUCTION ROLE	IMPACT ON THESE RESULTS	ADDRESSABLE LOCALLY?
llm-d inference layer (LLMInferenceService)	Production uses KServe LLMInferenceService v0.20.0 (a separate controller from classic KServe) with the llm-d endpoint picker (`llm-d-router-endpoint-picker v0.9.0`). The picker does cache-aware routing across GPU pods and disaggregated prefill/decode. Classic KServe is not used in production for LLM serving.	Test used LiteLLM proxying Ollama. Classic KServe is running on the local cluster (Ollama InferenceService in ai-platform), but the LLMInferenceService CRD and llm-d endpoint picker are not installed. No cache-aware routing or disaggregated serving in this test.	Partial - classic KServe already present. Installing llmops-kserve (the LLMInferenceService controller + llm-d) requires the additional chart from the repo. Medium effort, low disruption risk since it adds new CRDs alongside existing ones.
Envoy Gateway + Agent Router	Single front door, EKS token auth, routing to `/api/a2a/kagent/<name>/` . The AI Gateway ext_proc sidecar counts tokens per request and writes to Valkey quota counters.	Agents called via local Cilium gateway. No auth, no quota enforcement, no token counting at the gateway layer.	No - llmops-gateway chart needs Terraform-provisioned AWS resources (Transit Gateway, VPC endpoints, EKS Pod Identity).
OPA	Two enforcement points: (1) agent access - checks caller's tenant registry for the agent name, 403 otherwise; (2) model access - checks agent's model calls against the tenant's allowed models list. Runs as an Envoy ext_authz filter, reloads registry within ~1 minute of a change.	No enforcement in this test. Any caller can reach any agent and any model. Token usage is not charged to a tenant quota.	Partial - OPA can be deployed locally with the tenant registry, but without the Envoy gateway routing through it, it won't be in the request path.
Valkey (ElastiCache) quota counters	Every response is charged to the tenant's token quota in Valkey. The AI Gateway ext_proc writes the token cost; the quota check blocks requests over-limit before they reach the model.	No quota enforcement. No per-tenant usage accounting. Test ran uncapped against a shared LiteLLM instance.	Partial - Valkey (Redis-compatible) can run locally. Without the AI Gateway ext_proc writing to it, quota enforcement is not in the request path.

MISSING COMPONENT	PRODUCTION ROLE	IMPACT ON THESE RESULTS	ADDRESSABLE LOCALLY?
vClusters **(bridged)**	One virtual cluster per tenant, running as pods in the host namespace. Gives each tenant an isolated Kubernetes API server - they cannot see other tenants or host cluster internals. Agents live on the host cluster; tenants call them from inside their vCluster via the gateway.	Bridged - vCluster v0.37.6 installed, tenant vCluster `cappuccino` running in `vcluster-cappuccino`. Called `baseline-go-minimal` from inside: state completed, 639/115 tokens. Namespace isolation confirmed - only default/kube-system visible from inside. Remaining gap: EKS OIDC JWT auth and the vCluster token injection (MutatingAdmissionPolicy) are not configured.	Done - `vcluster create cappuccino --namespace vcluster-cappuccino`. Network connectivity to host cluster gateway works without additional config.
AWS Bedrock - Claude Sonnet 5 **(bridged)**	Production model is `us.anthropic.claude-sonnet-5` via Bedrock regional inference profile. Reliable tool-call schema compliance (no ask_user misfires). Latency is lower than local Ollama.	Bridged with `claude-sonnet-5-5` via the Anthropic API through LiteLLM. Zero tool-call misfires. Token counts and cost estimates in this test use real Sonnet 5.5 rates ($2.00/$10.00 per MTok). Not identical to a managed Bedrock inference profile, but model behaviour and cost profile are equivalent for this test.	Done - LiteLLM ModelConfig updated to `claude-sonnet-5-5`. ANTHROPIC_API_KEY injected via secret. Remaining gap: Bedrock IAM + regional inference profile routing not tested.
llmops-agents chart	Hardened kagent: external Postgres for agent state, namespaced RBAC, apiKeyPassthrough (caller's EKS token forwarded to the model gateway).	Vanilla upstream kagent chart used. No external database, no caller token passthrough. Agent state lives in the controller's in-cluster store.	Yes - chart is in the repo. llmops-agents wraps upstream kagent with an external Postgres database (already running via kagent-postgresql). Medium effort to swap in. ~1-2 hours.
Karpenter + GPU nodes (EC2 G6e / L40S)	GPU node autoscaling for vLLM. DCGM exporter provides GPU metrics via NVIDIA GPU Operator v26.7.0.	Not relevant for kagent agent pods (CPU-only). Model serving latency in this test reflects local GPU hardware, not EC2 G6e instance performance.	N/A - requires EKS and AWS GPU instances.

These results establish a kagent agent framework overhead baseline - pod footprint, token throughput, and latency per agent type. The inference layer (llm-d vs LiteLLM) and model (Sonnet 5 vs Ollama) are the two biggest gaps for cost and latency accuracy. Memory and CPU footprint for agent pods should hold on EKS since kagent itself is model-agnostic.

Production Gateway Architecture

The Envoy Gateway enforces three things in sequence for every request:

1. Identity

The caller presents a Kubernetes ServiceAccount token. The gateway verifies it and strips any identity headers the client tries to inject. Tenants in vClusters get the token automatically.

2. Authorisation

OPA checks whether this tenant is allowed to call this agent or model. Tenant permissions are managed in a config file and update live without restarting the gateway.

3. Quota

Token usage is counted per response and charged against the tenant's quota in a shared counter store. Requests from tenants over their limit are blocked before reaching the model.